

# Práctica REST con Spring

## Desarrollo de Aplicaciones Web (3º Gr. Ing. Inf)

### Práctica 1: Desarrollo de un Servicio Web REST

El objetivo de esta práctica es desarrollar un servicio Web REST utilizando Spring Boot que permita gestionar una colección de canciones MP3. La aplicación no utilizará base de datos; los datos se almacenarán en memoria (por ejemplo, en `List` o `Map`). El servicio deberá implementar varios recursos REST, cada uno con una serie de métodos aplicables que permitan consultar y modificar la colección. A continuación se detalla el formato del DTO (*Data Transfer Object*) que representa una canción, los recursos y los métodos aplicables.

#### Data Transfer Object

El servicio gestiona canciones y cada canción tiene los siguientes atributos:

- `id` (entero largo)
- `titulo` (cadena de caracteres)
- `cantante` (cadena de caracteres)
- `anio` (entero)

Los datos deben intercambiarse en **JSON**.

#### Recursos

El servicio gestiona la colección completa de canciones, las canciones particulares de dicha colección y la lista de canciones de un cantante especificado. Cada uno de estos recursos está representado por una URI (*endpoint*). Abajo se detallan las URIs relativas al servicio y los métodos que se pueden aplicar a ellas junto con una descripción de su semántica.

| Recurso                            | Método | Descripción                                                                                                                                                                                                                                                                                              |
|------------------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/canciones</code>            | GET    | Devuelve la lista completa de canciones. El código de respuesta es siempre 200 (OK).                                                                                                                                                                                                                     |
| <code>/canciones</code>            | POST   | Añade una nueva canción a la lista. La nueva canción se debe pasar en el cuerpo de la petición. Debe devolver 201 ( <i>Created</i> ) como código de respuesta y contener la cabecera <code>Location</code> [1] (ver nota abajo) con la URI que representa a la nueva canción.                            |
| <code>/canciones/{id}</code>       | GET    | Consulta la canción con el identificador indicado. Si lo encuentra debe devolverlo con un código de respuesta 200 (OK), en caso contrario debe devolver 404 ( <i>Not found</i> ).                                                                                                                        |
| <code>/canciones/{id}</code>       | PUT    | Actualiza la canción con el identificador indicado. La nueva versión de la canción debe pasarse en el cuerpo de la petición. Si puede hacer la actualización debe devolver la nueva versión de la canción con un código de respuesta 200 (OK), en caso contrario debe devolver 404 ( <i>Not found</i> ). |
| <code>/canciones/{id}</code>       | DELETE | Eliminar la canción con el identificador indicado. Si la canción existe debe devolver un código de respuesta 204 ( <i>No content</i> ) y el cuerpo vacío, en caso contrario debe devolver 404 ( <i>Not found</i> ).                                                                                      |
| <code>/canciones?cantante=x</code> | GET    | Devuelve una lista de canciones cuyo cantante es el indicado. El código de respuesta es siempre 200 (OK).                                                                                                                                                                                                |

[^1] Ejemplo de implementación de Location en Spring:

```
public ResponseEntity<Cancion> create(Cancion cancion,
                                     UriComponentsBuilder uriComponentsBuilder) {
    Cancion creada = service.create(cancion);
    URI location = uriComponentsBuilder.path("/canciones/{id}")
                                       .buildAndExpand(creada.getId())
                                       .toUri();
    return ResponseEntity.created(location).body(creada);
}
```

## Ejemplos

En esta sección se presentan una serie de ejemplos que contienen el comando `curl` para probarlo y la respuesta que debería dar el servicio.

### Ejemplo 1. Obtener todas las canciones

```
curl -i --noproxy localhost http://localhost:8080/canciones
```

#### Respuesta

HTTP/1.1 200 OK

Content-Type: application/json

```
[
  {
    "id": 1,
    "titulo": "Mediterráneo",
    "cantante": "Jose Manuel Serrat",
    "anio": 1971
  },
  {
    "id": 2,
    "titulo": "Hijo de la Luna",
    "cantante": "Mecano",
    "anio": 1986
  }
]
```

Si no hay canciones:

```
[]
```

---

### Ejemplo 2. Añadir una canción

```
curl -i --noproxy localhost -X POST http://localhost:8080/canciones
-H "Content-Type: application/json" -d '{
  "titulo": "Como Camarón",
  "cantante": "Estopa",
  "anio": 1999
}'
```

#### Respuesta

Suponiendo que se han añadido otras dos canciones antes, la respuesta sería:

HTTP/1.1 201 Created

Content-Type: application/json

Location: http://localhost:8080/canciones/3

```
{
  "id": 3,
  "titulo": "Como Camarón",
  "cantante": "Estopa",
  "anio": 1999
}
```

---

### Ejemplo 3. Obtener una canción concreta

```
curl -i --noproxy localhost http://localhost:8080/canciones/1
```

**Respuesta correcta**

HTTP/1.1 200 OK

```
{
  "id": 1,
  "titulo": "Mediterráneo",
  "cantante": "Jose Manuel Serrat",
  "anio": 1971
}
```

**Canción inexistente**

HTTP/1.1 404 Not Found

---

### Ejemplo 4. Actualizar una canción

```
curl -i --noproxy localhost -X PUT http://localhost:8080/canciones/1
-H "Content-Type: application/json" -d '{
  "titulo": "Mediterráneo",
  "cantante": "Jose Manuel Serrat",
  "anio": 1972
}'
```

**Respuesta correcta**

HTTP/1.1 200 OK

```
{
  "id": 1,
  "titulo": "Mediterráneo",
  "cantante": "Jose Manuel Serrat",
  "anio": 1972
}
```

**Canción inexistente**

HTTP/1.1 404 Not Found

---

### Ejemplo 5. Eliminar una canción

```
curl -i --noproxy localhost -X DELETE http://localhost:8080/canciones/1
```

**Respuesta correcta**

HTTP/1.1 204 No Content

## Canción inexistente

HTTP/1.1 404 Not Found

---

## Ejemplo 6. Buscar canciones por cantante

```
curl -i --noproxy localhost http://localhost:8080/canciones?cantante=Estopa
```

### Respuesta

HTTP/1.1 200 OK

```
[
  {
    "id": 3,
    "titulo": "Como Camarón",
    "cantante": "Estopa",
    "anio": 1999
  }
]
```

### Si no hay resultados

```
[]
```

---